Estrutura de dados e organização de arquivos

Prof. David Alves

14/10/2026

Árvores Balanceadas (AVL)

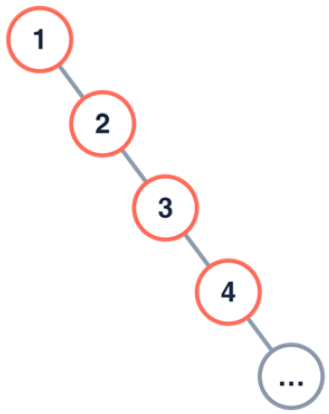
Conceitos, Fator de Balanceamento e Rotações

Slide 2: O que é uma árvore AVL?
Definição: É uma Árvore Binária de Busca (ABB) **balanceada**.
Origem: Criada por Adelson-Velsky e Landis (1962).
Propriedade Fundamental: Para qualquer nó da árvore, a altura da subárvore esquerda e a altura da subárvore direita diferem, no máximo, por **1**.
Objetivo: Evitar que a árvore fique em uma estrutura linear (formato de lista), garantindo complexidade de tempo garantida de $O(\log n)$ para buscas, inserções e remoções.

As operações básicas, tais como busca, inserção e remoção, levam tempo proporcional ao número de níveis da árvore binária de busca.

É importante manter a árvore com a menor quantidade de níveis possível.

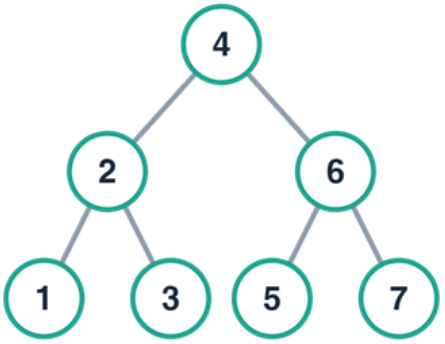
exemplo, inserção dos números de 1 a 10



neste caso, n elementos = n níveis = péssima distribuição

idealmente

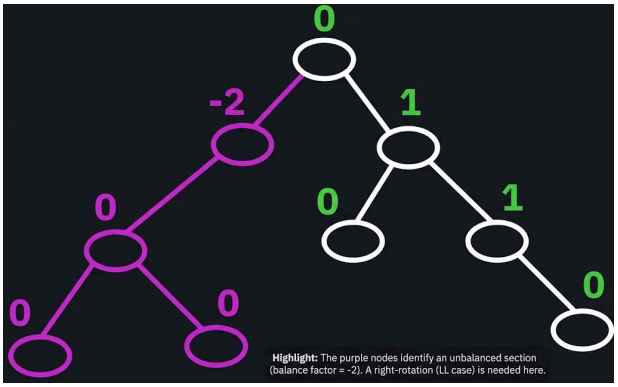
árvore equilibrada com 3 níveis



Como garantir o equilíbrio?
Resposta: Com rotações

Dividindo o problema em 2 partes:
1 - Detectar o desequilíbrio (calcular o fator de equilíbrio)
2 - Corrigir o desequilíbrio

O equilíbrio de uma árvore binária de busca é medido subtraindo o número de níveis da subárvore da esquerda do número de níveis da subárvore da direita.
Exemplo:



a subárvore rosa está desequilibrada!
Uma subárvore está desequilibrada apenas quando este número for maior que 1 ou menor que -1.

O desequilíbrio da árvore é corrigido através das rotações.
Existem 4 tipos básicos de rotação:
2 tipos primitivos
* Rotação à esquerda
* Rotação à direita
2 tipos compostos
* Rotação dupla à esquerda
* Rotação dupla à direita

O Fator de Balanceamento (Q)

- **Conceito:** Mede o nível de desequilíbrio de um nó comparando as alturas de suas sub-árvores.
- **Fórmula Prática:**
    - $Q = R - L$
- *(Onde R representa a altura da sub-árvore da direita e L a altura da sub-árvore da esquerda).*
- **Critério de Equilíbrio:**
    - Se $-1 \leq Q \leq 1$: A árvore está **equilibrada**.
    - Se $Q > 1$ ou $Q <- 1$: A árvore está **desequilibrada** e exige rotações.

Como Decidir as Rotações

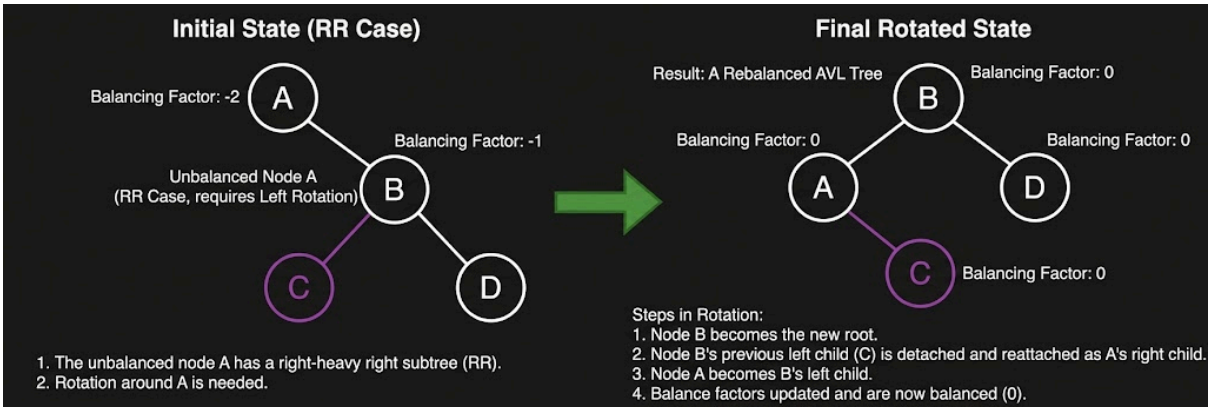
Esquema prático demonstrado na aula para tomada de decisão:

- **Se $-1 \leq Q \leq 1$:**
    - → **Árvore Equilibrada** (Nenhuma ação necessária).
- **Se $Q > 1$ (Desequilíbrio à Direita):**
    - Se a sub-árvore da direita tem $Q < 0$ → **Rotação Dupla à Esquerda** (Direita-Esquerda).
    - Senão ($Q \geq 0$) → **Rotação Simples à Esquerda**.
- **Se $Q <- 1$ (Desequilíbrio à Esquerda):**
    - Se a sub-árvore da esquerda tem $Q > 0$ → **Rotação Dupla à Direita** (Esquerda-Direita).
    - Senão ($Q \leq 0$) → **Rotação Simples à Direita**.

Os 4 Tipos de Rotações

- **Rotação Simples à Direita (LL):** Corrige desequilíbrios à esquerda na sub-árvore esquerda.
- **Rotação Simples à Esquerda (RR):** Corrige desequilíbrios à direita na sub-árvore direita.
- **Rotação Dupla à Esquerda (RL / Direita-Esquerda):** Aplicada quando há um "joelho" à direita-esquerda.
- **Rotação Dupla à Direita (LR / Esquerda-Direita):** Aplicada quando há um "joelho" à esquerda-direita.

Rotação à esquerda.
O filho da direita vira a nova raiz
A raiz original vira o filho da esquerda da nova raiz
Quando o filho da direita já tem um filho da esquerda,
O filho da esquerda do filho da direita vira o filho da direita do filho da esquerda

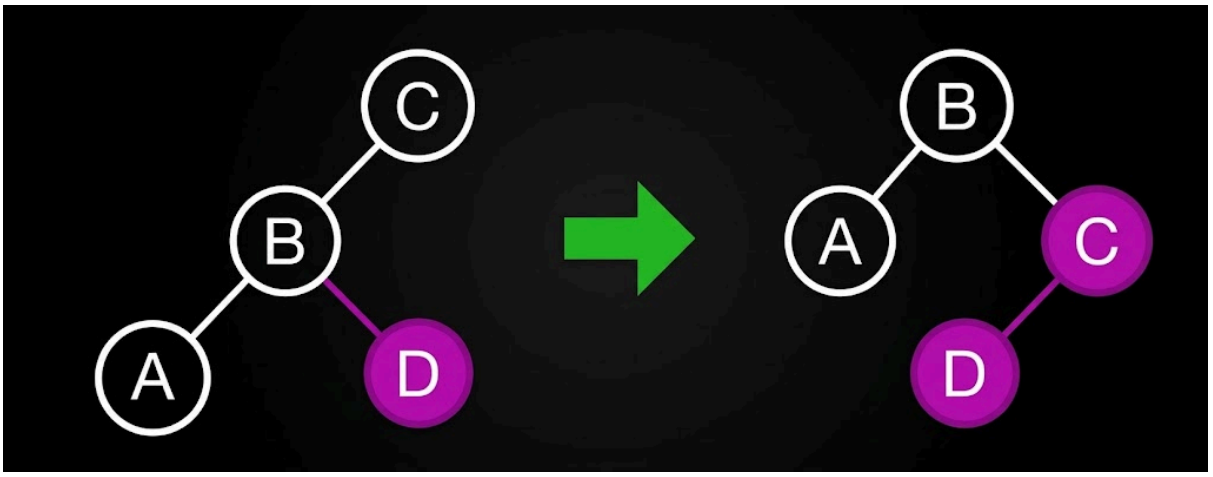


Rotação à direita

O filho da esquerda vira a nova raiz

O filho da direita de B vira o filho da esquerda de quem era a antiga raiz

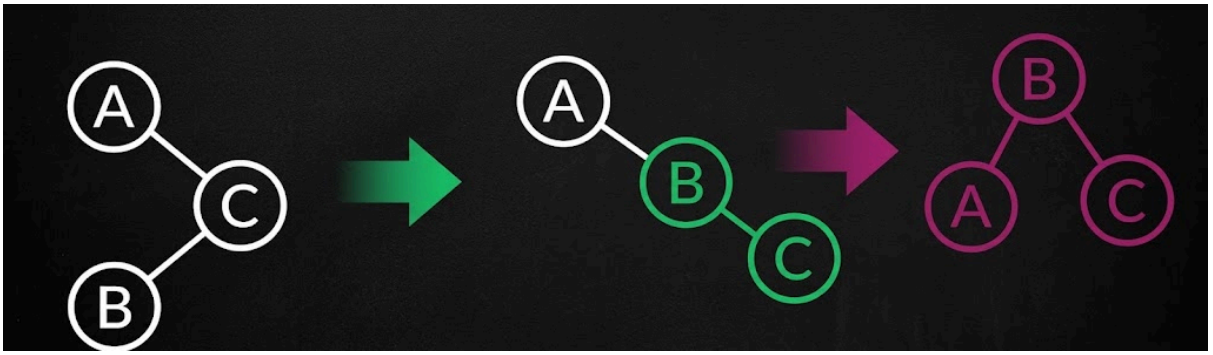
C vira filho da direita da nova raiz.



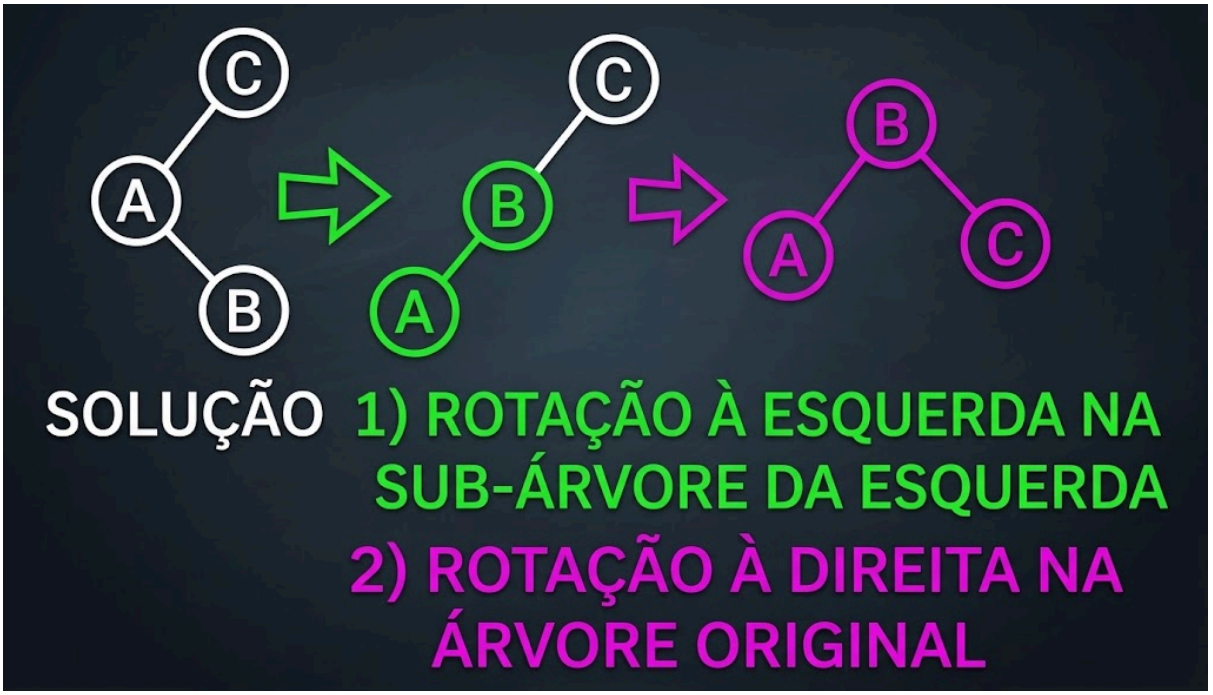
Rotação dupla à esquerda



Desequilíbrio positivo na raiz original
Desequilíbrio negativo na subárvore da direita
Neste caso a rotação simples não resolve o problema
então fazemos uma rotação à direita na subárvore da direita
e uma rotação à esquerda na árvore original



Rotação dupla à direita
Rotação à esquerda na subárvore da esquerda
Rotação à direita na árvore original



Vantagens e Aplicações

- **Vantagem Principal:** Otimização severa de desempenho, mantendo a altura estritamente controlada mesmo após milhares de inserções.
- **Aconselhado para:** Sistemas onde operações de **busca** são muito mais frequentes do que inserções/remoções.
- **Aplicações Comuns:**
    - Indexação em Bancos de Dados.
    - Sistemas de arquivos.
    - Tabelas de símbolos em compiladores.

Conclusão

- As árvores AVL resolvem o problema clássico de pior caso das ABBs tradicionais.
- O controle rigoroso por meio do fator Q mantém a árvore compacta e eficiente.